

SQL Fundamentals

Query Any Database with Confidence

■■ DATABASE QUERYING

■ LEARNING OUTCOME

By the end of this PDF, you will write SQL queries to extract, filter, group, and join data from relational databases — the core skill of every data analyst.

■ Skills You Will Gain

- Write SELECT queries with filters and sorting
- Use GROUP BY for aggregations
- Join multiple tables with INNER, LEFT, RIGHT JOIN
- Use subqueries and CTEs
- Apply window functions for advanced analytics
- Understand database design basics

■ SQL is required in 75% of Data Analyst job postings. It is often more valuable than Python for business analytics roles.

SECTION 1

SQL Basics — SELECT Statement

```
-- Select all columns SELECT * FROM employees; -- Select specific columns SELECT name, salary, department FROM employees; -- Filter with WHERE SELECT * FROM employees WHERE salary > 50000; -- Sort results SELECT name, salary FROM employees ORDER BY salary DESC; -- Limit results SELECT * FROM employees LIMIT 10;
```

SECTION 2

Filtering Data

```
-- Multiple conditions SELECT * FROM orders WHERE status = 'Shipped' AND amount > 1000; -- Range filter SELECT * FROM products WHERE price BETWEEN 100 AND 500; -- List filter SELECT * FROM employees WHERE department IN ('Sales', 'Marketing'); -- Pattern matching SELECT * FROM customers WHERE name LIKE 'A%'; -- Null check SELECT * FROM orders WHERE delivery_date IS NULL;
```

SECTION 3

Aggregations & GROUP BY

```
-- Count, Sum, Avg, Max, Min SELECT department, COUNT(*) AS employee_count, AVG(salary) AS avg_salary, MAX(salary) AS highest_pay FROM employees GROUP BY department HAVING AVG(salary) > 40000 ORDER BY avg_salary DESC;
```

WHERE filters individual rows BEFORE grouping. HAVING filters groups AFTER GROUP BY is applied.

SECTION 4

Joining Tables

JOIN Type	Returns
INNER JOIN	Only rows that match in BOTH tables
LEFT JOIN	All rows from left table + matching from right (NULL if no match)
RIGHT JOIN	All rows from right table + matching from left
FULL OUTER JOIN	All rows from both tables

```
SELECT o.order_id, c.customer_name, o.total_amount FROM orders o INNER JOIN customers c ON o.customer_id = c.id WHERE o.total_amount > 5000 ORDER BY o.total_amount DESC;
```

SECTION 5

Subqueries & CTEs

Subquery (query inside a query):

```
SELECT name, salary FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);
```

CTE (Common Table Expression) — cleaner and reusable:

```
WITH dept_avg AS ( SELECT department, AVG(salary) AS avg_sal FROM employees GROUP BY department ) SELECT e.name, e.salary, d.avg_sal FROM employees e JOIN dept_avg d ON e.department = d.department WHERE e.salary > d.avg_sal;
```

SECTION 6

Window Functions

```
-- Rank employees by salary within department SELECT name, department, salary, RANK() OVER (PARTITION BY department ORDER BY salary DESC) AS rank, ROW_NUMBER() OVER (ORDER BY salary DESC) AS global_rank, SUM(salary) OVER (PARTITION BY department) AS dept_total FROM employees;
```

■ PRACTICE QUESTIONS

Q1. Write a query to find all products with price > 500, sorted by price descending.

*Answer: SELECT * FROM products WHERE price > 500 ORDER BY price DESC;*

Q2. How do you count the number of orders per customer?

Answer: SELECT customer_id, COUNT() AS order_count FROM orders GROUP BY customer_id;*

Q3. What is the difference between WHERE and HAVING?

Answer: WHERE filters rows before grouping; HAVING filters groups after GROUP BY.

Q4. Write a query to get names of employees earning above average salary.

Answer: SELECT name FROM employees WHERE salary > (SELECT AVG(salary) FROM employees);

Q5. When would you use a LEFT JOIN instead of INNER JOIN?

Answer: When you want ALL rows from the left table, even if there's no match in the right table.

■ MINI PROJECT

Apply what you've learned with a real dataset!

Dataset: Tables: orders(id, customer_id, date, amount), customers(id, name, city), products(id, name, category, price)

Steps:

1	Count Total orders per city
2	Revenue Total revenue by product category
3	Top 5 Top 5 customers by total spend

4	Monthly Monthly revenue trend for last 12 months
5	Join Show order details with customer name and product

Expected Output: 5 SQL queries + results table for each

Insights to Find: Best city, top category, most valuable customers, revenue growth

■■ SQL Quick Reference Cheat Sheet	
<code>SELECT col FROM tbl</code>	Basic query
<code>WHERE col = 'value'</code>	Filter rows
<code>ORDER BY col DESC</code>	Sort results
<code>LIMIT n</code>	Return first n rows
<code>GROUP BY col</code>	Aggregate by group
<code>HAVING condition</code>	Filter groups
<code>COUNT(*)</code>	Count rows
<code>SUM/AVG/MAX/MIN(col)</code>	Aggregate functions
<code>INNER JOIN tbl ON cond</code>	Match both tables
<code>LEFT JOIN tbl ON cond</code>	All left + matched right
<code>IS NULL / IS NOT NULL</code>	Check for missing values
<code>LIKE 'A%'</code>	Pattern: starts with A
<code>IN ('a','b','c')</code>	Match list of values
<code>BETWEEN x AND y</code>	Range filter
<code>WITH cte AS (...)</code>	Define reusable subquery